

Type 1 · Function with a Formula

TEMPLATE · e.g. Q1

The question gives formulas, and the function **returns** the result.

```
def function_name(param1, param2, param3):  
    step1 = <formula using params>  
    step2 = <formula using params>  
    result = <formula using step1, step2>  
    return result  
  
a = float(input("Enter ...: "))  
b = float(input("Enter ...: "))  
c = float(input("Enter ...: "))  
  
answer = function_name(a, b, c)  
print()  
print("Label:", round(answer, 2), "<unit>")
```

Type 1 · Function with a Formula

CHECKLIST

- Use the **exact function name and parameter order** from the question.
- Use `float(input())` for decimals and `int(input())` for whole numbers (ages, counts).
- The function must return the result. Don't `print` inside it unless the question asks you to.
- Only use `round(x, 2)` if the question says "rounded to 2 decimal places".
- Write the function first, then the test code that asks for input, calls the function and prints the result.

Type 2 · Conditions / Categories

TEMPLATE · e.g. Q2

```
def function_name(value, flag):
    # 1. Special cases that skip everything else go first
    if <special case>:
        return <fixed value>
    # 2. Pick the category, from lowest range to highest
    if value <= <limit1>:
        result = <amount1>
    elif value <= <limit2>:
        result = <amount2>
    else:
        result = <amount3>

    # 3. Apply extra rules (surcharge, discount...)
    if flag == "yes":
        result = result + <extra>
    return result

value = int(input("Enter ...: "))
flag = input("Question? ")
answer = function_name(value, flag)
print()
print("Label: $" + str(answer))
```

Type 2 · Conditions / Categories

CHECKLIST

- Handle the **exception first** and return early (for example, "children under 5 are always free").
- With `elif`, each condition only needs the **upper limit**, because earlier branches have already caught the lower values.
- Check the boundary words carefully. "Under 5" means `< 5`, "5–17" means `<= 17`, and "65 or older" means `else` or `>= 65`.
- Compare text with `== (flag == "yes")`. Don't convert it with `int()`.
- Print money as `"Label: $" + str(answer)` so there's no space after the \$.

Type 3 · Loop with Totals & Counter

TEMPLATE · e.g. Q3

```
total = 0.0          # running total, starts at 0
total_cost = 0.0     # second running total, if needed
count = 0            # counter, starts at 0

for i in range(1, N + 1):          # 1 to N: "day 1", "day 2"...
    value = float(input("Enter value for item " + str(i) + ": "))

    if value <= <limit>:
        cost = value * <rate1>
    else:
        cost = value * <rate2>
        count = count + 1          # count items meeting the condition

    total = total + value          # add to the running totals
    total_cost = total_cost + cost

print()
print("Total:", total)
print("Count:", count)
print("Total cost: $" + str(round(total_cost, 2)))
# average = total / N              # if the question asks for an average
```

Type 3 · Loop with Totals & Counter

CHECKLIST

- **Start every total at 0 before the loop**, never inside it.
- Add to the totals **inside** the loop, and print **after** it, without indentation.
- No `sum()`, `max()` or `min()`. Do the math by hand with `total = total + value`.
- To find the **max or min** by hand, set the first value as the starting max, then compare with `if value > biggest: biggest = value`.
- `range(1, N + 1)` counts 1 to N. `range(N)` counts 0 to N-1.

Type 4 · Nested-Loop Pattern

TEMPLATE · e.g. Q4

```
n = int(input("Enter a positive integer: "))
print()

for row in range(n):          # outer loop = rows
    for col in range(<width>): # inner loop = items on each row
        print(<what to print>, end=" ")
    print()                  # go to the next line after each row
```

- Use `end=" "` to keep numbers on one line, and an empty `print()` to end each row.
- `row % 2 == 0` is true for even rows (0, 2, 4...), so use it for anything that alternates.
- Check your loop with the smallest case, `n = 1` or `2`.

Type 4 · Nested-Loop Pattern

SHAPES TABLE

Swap these into the template on the previous card:

Pattern	<width>	<what to print>
Square of *	n	""
Alternating rows 0/1	n	0 if row % 2 == 0 else 1
Alternating columns 0/1	n	col % 2
Checkerboard 0/1	n	(row + col) % 2
Triangle growing (1, 2, 3...)	row + 1	""
Triangle shrinking	n - row	""
Number triangle 1 / 1 2 / 1 2 3	row + 1	col + 1
Each row repeats its number	row + 1	row + 1

- Put each question in its own section with a header comment.
- If the question says "define a function", write the function first, then the test code that calls it.
- Make the `input()` prompts and `print()` labels match the sample run.
- Test with the sample input, then with **another input** (the program must work for any valid value).
- Don't use `sum`, `max`, `min` or `pow` unless the question allows them. Write the logic with loops and `if` statements.
- Allowed: `print()`, `input()`, `int()`, `float()`, `range()`, `round()`.